HARNESS LAB · MÓDULO 1 · AULA 1.5

Escolha sua *feature-alvo*

A feature do seu repositório que o loop vai entregar no dia 7. Ela não precisa impressionar ninguém: precisa passar em cinco peneiras e ter número na porta.

`FEATURE-ALVO.md` `cinco peneiras` `critério de aceite` `Quando ..., então ...` `git grep -il` `plan mode` `/feature-alvo` `.git/info/exclude`

VOCÊ SAI COM

O `FEATURE-ALVO.md` na raiz do seu repositório: uma feature que passou nas cinco peneiras e de 3 a 5 critérios de aceite em rascunho, no formato de onde o Módulo 4 parte.

Material de apoio · Lucas Souza · Beer and Code

1 A aula em *uma página*

Última aula do Módulo 1. Você já entendeu por que o agente esquece a regra e separou as suas correções nas quatro camadas. Falta uma coisa: escolher o que o seu loop vai entregar no fim do curso. É a **feature-alvo**, e a escolha pesa mais do que parece.

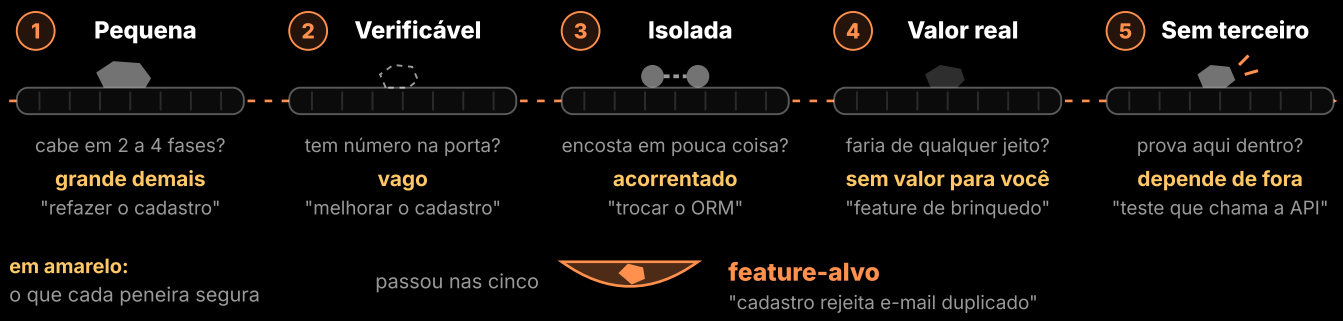
Endereço vago sempre parece entregue

Com "perto da praça", o entregador deixa o pacote numa casa qualquer e marca entregue. Ninguém consegue provar que errou. É o pedido "melhorar o cadastro": ele mexe em alguma coisa e diz pronto. Com "Rua 7, nº 120", ele para no 118, o número não bate e o erro aparece. Errar ficou visível, por isso dá para corrigir. Feature-alvo boa tem número na porta.

No dia 7, o loop entrega uma feature sua

No Módulo 6 você dispara o loop que montou, numa feature do seu repositório, não num projeto de exemplo. A garantia do curso foi pensada em cima disso: um resultado que se confere na tela. Ou o loop entregou tarefa testada no seu repositório, ou o log mostra que não entregou. A feature-alvo é onde esse resultado aparece.

Por isso ela não pode ser qualquer uma, e também não precisa impressionar ninguém. Pegue algo real, do seu trabalho ou de um projeto engavetado, e ponha o chapéu de garimpeiro: o cascalho passa por **cinco peneiras**, cada uma segura um tipo de pedra, e o que chega no fundo é pouco, pequeno e sólido. Peneira que falta deixa a pedra dela cair na bateia: pular só empurra o problema para o dia 7.



- **Pequena.** O harness funciona em projeto do zero, mas o seu dia a dia é feature. Comece por uma pequena, para entender o mecanismo inteiro.
- **Verificável.** Um destino que dá para validar, com métrica ou com código: o número na porta.
- **Isolada.** Troca o pneu, não mexe no motor. Um endpoint, um campo, uma validação passam. Trocar o ORM, a autenticação ou o framework faz o loop mexer em arquivo que você não pediu e quebrar teste que nem é da feature. Régua: se os arquivos que falam da área são mais do que você revisaria com calma num PR, corte por enquanto.
- **Valor real.** Algo que você já iria fazer. Feature de mentira não atesta que você está resolvendo um problema seu.
- **Sem terceiro instável.** O termômetro com mau contato lê 36, 39, 36 na mesma pessoa. Teste que chama API externa, sandbox de pagamento ou serviço de outro time fica vermelho e verde sem mudança de código, e o loop lê o vermelho como "eu errei": mexe no código que estava certo. Prove a parte de dentro; a integração fica para depois.

Alvo ruim, alvo bom. Os três pares do vídeo estão na tabela da seção 2, com mais exemplos. O do cadastro que rejeita e-mail duplicado segue até o fim.

Critério é lista de mercado. Com o bilhete "coisas pro café", volta banana e biscoito, e não há onde marcar certo ou errado. Com a lista, você confere item por item e aponta o leite desnatado. Critério de aceite é isso: cada linha diz o que é certo, de um jeito que outra pessoa, ou um teste, confere sem te perguntar. "Melhorar o cadastro" é o bilhete; "quando eu cadastro com um e-mail que já existe, dá erro e nenhum usuário novo é criado" é a lista. O formato: **quando eu faço tal coisa, então acontece tal coisa que eu consigo ver**. Hoje é rascunho. No Módulo 4 vira especificação e teste, mesmo que o projeto não tenha teste nenhum hoje.

> *"Does it know what done looks like? A goal condition, a spec file, or a failing test it has to turn green."*
>
> Documentação oficial do Claude Code · Best practices, "Let Claude run without you", a primeira pergunta antes de deixar uma tarefa rodando sozinha (code.claude.com/docs/en/best-practices)

Agora no seu repositório. Abra o seu assistente de código, leve os slides das peneiras e peça: "eu quero encontrar uma feature-alvo a partir dessa peneira aqui. Me ajuda a encontrar". Na tela: o `git log`, o `git grep` do que você marcou para depois e o `git grep -il` da área. Você sai com o `FEATURE-ALVO.md` com 3 a 5 critérios. O passo a passo está na seção 3.

A feature viaja sete dias. Dia 1: uma ideia no `FEATURE-ALVO.md`. Módulo 4 (dias 3 e 4): critério de verdade e os primeiros testes. Módulo 5 (dias 5 e 6): tarefas pequenas. Módulo 6 (dia 7): o loop passa tarefa por tarefa pela Prova, e a que falha volta e tenta de novo. Pode entregar tudo, ainda estar rodando ou falhar (a aula 6.2 lê o log de cada desfecho), mas a peça no tabuleiro é a que você escolheu hoje. Módulo 1 fechado: correções nas quatro camadas, repositório escolhido, `FEATURE-ALVO.md` escrito.

2 Indo *além da aula*

Mais alvos, e a peneira em que *cada um cai*

A tabela prometida na aula. Os marcados com "(aula)" são os do vídeo: neles, o bom é outro alvo do mesmo tipo de projeto. Nos demais, o bom é uma fatia do mesmo desejo que passa nas cinco peneiras, não a única possível; quem cai em valor real troca de alvo. Está também em `plugins/feature-alvo/skills/feature-alvo/references/exemplos.md`, que a skill lê.

TIPO	ALVO RUIM	CAI EM	ALVO BOM
Backend	Refatorar a autenticação inteira (aula)	pequena, isolada	Exportar os pedidos do mês em CSV: um endpoint, um arquivo, confere linha a linha (aula)
Backend	Deixar a API mais rápida	verificável	Paginar `GET /pedidos`: 20 por página, com o total no cabeçalho
Backend	Integrar com o gateway de pagamento	sem terceiro	Calcular o total do pedido com cupom e frete; a chamada ao gateway fica para depois
Backend	Login com Google	sem terceiro	Criar ou vincular o usuário a partir dos dados que o Google devolve, com a resposta gravada no teste

TIPO	ALVO RUIM	CAI EM	ALVO BOM
Front-end	Deixar o dashboard mais bonito (aula)	verificável	Filtro por status na lista de pedidos, salvo na URL (aula)
Front-end	Migrar o design system inteiro	pequena, isolada	Os botões do formulário de cadastro usam o componente novo, com o mesmo texto e a mesma ação
Front-end	Melhorar a experiência do formulário	verificável	Formulário de endereço mostra o erro em cada campo e não perde o que foi digitado
Legado	Migrar tudo para microserviços (aula)	pequena, isolada	Cadastro rejeita e-mail duplicado, ignorando maiúsculas (aula)
Legado	Atualizar o framework para a última versão	isolada	Atualizar só o pacote de PDF de 2.x para 3.x, com o boleto gerado igual
Legado	Reescrever a emissão de nota fiscal	pequena, sem terceiro	Validar o dígito do CNPJ do cliente antes de emitir
IA	Pôr IA para responder o cliente	verificável, sem terceiro	Validar o JSON que volta do modelo contra o schema, com respostas gravadas no teste
Qualquer	Feature de brinquedo num repositório de exemplo	valor real	Um item do backlog do seu time, no seu repositório

O alvo ruim não está proibido para sempre: é grande, vago, acoplado ou dependente de fora demais para ser o primeiro trabalho do loop. O resto do bolo continua lá para depois.

ALÉM DA AULA · PEQUENA TEM RÉGUA: DE 3 A 5 CRITÉRIOS

A régua de bolso é de 2 a 4 fases, e os critérios também medem. No Módulo 4, o `/plan` da aula 4.3 (cópia do `commands/plan.md` do BC Harness 0.2.0) usa a contagem de critérios confirmados para escolher o tamanho do plano: até 3, `light`; de 4 a 10, `standard`; 11 ou mais, `complete`. Passou de 10 critérios? Não é pequena: corte uma fatia. O `confere.sh` do plugin reprova acima de 10.

Fonte: BC Harness 0.2.0, `commands/plan.md`, "Complexity tier": "use the confirmed AC count as proxy".

ALÉM DA AULA · O REPOSITÓRIO TAMBÉM PASSA POR PENEIRA

A feature-alvo mora num repositório, e os próximos módulos pedem coisas dele. Tem git com histórico. Você pode criar uma branch de trabalho e commitar nela: no Módulo 5, o pré-voo do `loop.sh` recusa `main`, `master` e HEAD solto, porque o loop commita sozinho. O app ou os testes rodam na sua máquina, sem VPN nem serviço de fora: no Módulo 4, o comando de teste do projeto vira o juiz do loop. E a política do time permite agente nesse código.

Fonte: `loop.sh` do Módulo 5 (pré-voo); `qual-comando.sh` da aula 4.1.

ALÉM DA AULA · A PENEIRA "ISOLADA" SE MEDE COM O GIT

`git grep -il termo` lista os arquivos versionados que citam o termo: com `vendor/` e `node_modules/` no `.gitignore` (o normal), eles ficam fora da conta. Legado com `vendor/` versionado: `git grep -il "email" -- . ':!vendor' | wc -l`. Use dois ou três nomes da área, em português e em inglês, e veja quantos são teste (zero é normal: os primeiros nascem no Módulo 4). O `area.sh` do plugin faz as duas contas, sem `vendor/` e `node_modules/`, e aponta chamada para fora.

Fonte: git-grep, "Look for specified patterns in the tracked files in the work tree" (git-scm.com/docs/git-grep).

ALÉM DA AULA · TERCEIRO INSTÁVEL INCLUI O MODELO DE IA

A resposta de um LLM também muda sem você mudar o código, e o teste que depende dela vira sorteio. A parte de dentro quase sempre existe: o cálculo, a validação, o payload que você monta, o tratamento de uma resposta gravada. No teste, o serviço de fora vira um falso: `Http::fake()` no Laravel (com `Http::preventStrayRequests()`, chamada sem resposta falsa vira exceção em vez de sair para a rede), `nock` no Node, `responses` no Python.

Fontes: Laravel, HTTP Client, "Testing" (laravel.com/docs/http-client#testing); github.com/nock/nock; github.com/getsentry/responses.

ALÉM DA AULA · COMMIT OU EXCLUDE, NUNCA SOLTO

No repositório da empresa, o `.git/info/exclude` ignora o arquivo só na sua máquina, sem mexer no `.gitignore` do time. Solto, fora do git e do exclude, ele atrapalha no Módulo 5: o pré-voo do `loop.sh` recusa árvore suja (o loop commita com `git add -A` e levaria o arquivo junto), e o `git clean -fd` que o loop sugere quando uma fase falha apaga arquivo não versionado. No exclude, só o `-x` ou o `-X` apagariam.

Fontes: gitignore, padrões "specific to one user's workflow" vão no `info/exclude` (git-scm.com/docs/gitignore); `loop.sh` do Módulo 5. Testado com `git clean -fdn`: solto, "Would remove FEATURE-ALVO.md"; no exclude, nada.

ALÉM DA AULA · O RASCUNHO QUE O MÓDULO 4 CONVERTE SEM RETRABALHO

Na aula 4.3, cada linha ganha um ID (`AC-01`), uma forma (`QUANDO` ou `SE` ... o sistema `DEVE`) e uma linha "Confere:". Escreva hoje pensando nisso: um comportamento por linha; um resultado que se lê (estado no banco, contagem, mensagem exata entre aspas, status); o caminho feliz que não pode quebrar também vira critério. Nada de palavra vaga (rápido, bom, adequado, amigável). E não numere com `AC-` agora: o `confere-phases.sh` do Módulo 5 lê todo `AC-NN` do arquivo como critério.

Fonte: `modelo-criterios.md` e `exemplo-cadastro/` da aula 4.3; `confere-phases.sh` do Módulo 5.

3 Faça *no seu repositório*

1 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou numa aula anterior? Atualize:

Terminal · em qualquer pasta — shell

```
git -C ~/harness-lab-materiais pull
```

2 Ponha o modelo do FEATURE-ALVO.md na raiz do seu projeto.

Cria o `FEATURE-ALVO.md` com seis seções, três critérios para preencher e as cinco peneiras. Já tem um? O seu fica intacto e o modelo vai para `FEATURE-ALVO.md.harness-lab-novo`.

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 1.5
```

3 Instale o plugin feature-alvo.

A skill `/feature-alvo`: as cinco peneiras embaladas para o Claude Code, em todos os seus projetos. O `update` importa se você adicionou o marketplace numa aula anterior: sem ele, o install não acha o plugin novo.

Terminal · em qualquer pasta — claude

```
claude plugin marketplace add beerandcodeteam/harness-lab-materiais
claude plugin marketplace update harness-lab
claude plugin install feature-alvo@harness-lab
claude plugin list
```

Ou, dentro do Claude Code:

Dentro do Claude Code — sessão

```
/plugin marketplace add beerandcodeteam/harness-lab-materiais
/plugin marketplace update harness-lab
/plugin install feature-alvo@harness-lab
/reload-plugins
```

4 Escolha a feature com o Claude Code.

Dentro do Claude Code · na raiz do projeto — sessão

```
/feature-alvo
/feature-alvo exportar os pedidos do mês em CSV
```

A skill levanta candidatas reais (commits, branches, marcadores no código, o backlog que você contar), passa cada uma nas cinco peneiras, mede a área, espera você escolher, rascunha os critérios, preenche o `FEATURE-ALVO.md` e roda o `confere.sh`. Só escreve esse arquivo, sem commit; no modo de plano, grava quando você aprovar. Depois da sua primeira resposta, o Claude Code pode pedir para rodar o `area.sh` e o `confere.sh`: são do plugin e só leem. Aprove.

5

Sem o plugin: o pedido da aula, no modo de plano.

Abra o `claude`, Shift+Tab até `plan mode on` (ou `claude --permission-mode plan`) e cole o pedido. Ele lê e roda comando para explorar, mas não grava até você aprovar o plano. Valor real é pergunta para você, não resposta dele; candidata sem evidência é invenção. Também em `modelos/pedido-feature-alvo.md`, com o que conferir na resposta.

PEDIDO · COLE NO CLAUDE CODE

```
Quero escolher a feature-alvo do Harness Lab neste repositório: a feature que um loop vai entregar sozinho no fim do curso. Me ajude a encontrar.

1. Leia o repositório sem alterar nada: git log --oneline -20, as branches e os marcadores que deixei para depois no código (FIXME, HACK e afins). Me pergunte o que está no backlog do time.

2. Monte 3 candidatas reais, cada uma com a evidência (commit, arquivo:linha ou o que eu contei).

3. Passe cada uma nas cinco peneiras, nesta ordem, e diga em qual cai: pequena (cabe em 2 a 4 fases); verificável (dá para escrever critério "Quando ..., então ..." que um teste confere); isolada (quantos arquivos falam da área, com git grep -il); valor real (pergunte para mim); sem terceiro instável (a prova não depende de API externa, sandbox de pagamento ou serviço de outro time). Se cair, proponha a fatia que passa.

4. Eu escolho. Depois rascunhe de 3 a 5 critérios "Quando ..., então ..." com resultado que dá para conferir e preencha o FEATURE-ALVO.md da raiz, com os títulos que já estão nele. Não mexa em mais nada.
```

Fonte: permission-modes, "Plan mode tells Claude to research and propose changes without making them." (code.claude.com/docs/en/permission-modes)

6

Ou olhe você mesmo, no terminal.

O que você mexeu por último e ficou pela metade, o que você marcou para depois (sem `-i` de propósito: em português, "todo" e "método" entrariam; `-w` pega a palavra inteira) e a área da candidata: troque `email` pela palavra da sua área e tente sinônimos.

Terminal · na raiz do seu projeto — shell

```
git log --oneline -20
git grep -nwE "TODO|FIXME|HACK" | head -20
git grep -il "email" | wc -l
git grep -il "email"
```

7

Confira que nenhum espaço do modelo sobrou.

Conta os `<...>` do modelo fora de crases: recém-instalado dá 7, pronto precisa dar **0**. Código ou tag de verdade vai entre crases. A conferência completa é o `confere.sh`, no fim da skill.

Terminal · na raiz do seu projeto — shell

```
sed 's/`[^`]*`//g' FEATURE-ALVO.md | grep -cE "<[^>]+>"
```

8 **Decida: commit ou exclude.**

Repositório seu: commit.

Terminal · na raiz do seu projeto — shell

```
git add FEATURE-ALVO.md
git commit -m "docs: feature-alvo do Harness Lab"
```

Repositório da empresa, sem versionar: o arquivo fica só na sua máquina e some do `git status`.

Terminal · na raiz do seu projeto — shell

```
echo "/FEATURE-ALVO.md" >> .git/info/exclude
git status --short
```

O modelo *instalado*

O `FEATURE-ALVO.md` do `instalar.sh 1.5` tem seis seções, na ordem de onde o Módulo 4 parte: Descrição, Por que é pequena, Critérios de aceite (rascunho), Como eu verificaria hoje na mão, Riscos e Cinco peneiras. Cada `<...>` é um espaço para o seu texto; as peneiras vêm sem marcar.

O exemplo da aula, *preenchido*

Trecho de `modelos/FEATURE-ALVO.exemplo.md`, o mesmo arquivo de onde a aula 4.3 parte. Repare: cada critério tem um resultado que um teste lê, e o último protege o caminho feliz. Os riscos do arquivo dizem o que fica fora: convite, importação por CSV e login social.

modelos/FEATURE-ALVO.exemplo.md (trecho) — exemplo para ler · não é instalado

```
## Critérios de aceite (rascunho)
- [ ] Quando cadastro com um e-mail que já existe, então recebo erro
      de validação e nenhum usuário novo é criado.
- [ ] Quando cadastro Ana@Exemplo.com e já existe ana@exemplo.com,
      então o cadastro é recusado do mesmo jeito.
- [ ] Quando o cadastro é recusado, então a mensagem "Este e-mail já
      está cadastrado" aparece abaixo do campo de e-mail.
- [ ] Quando cadastro um e-mail novo, então o usuário é criado e o
      login funciona como antes.
```

4 Arquivos que *reforçam a aula*

`FEATURE-ALVO.md`

O modelo: seis seções, três critérios "Quando ..., então ..." e as cinco peneiras como checklist. Mesmos títulos do Módulo 4, sem `AC-NN`.

No repositório: modulo-1-o-operador-e-o-trilho/aula-1.5-escolha-sua-feature-alvo/projeto/

`modelos/FEATURE-ALVO.exemplo.md`

O exemplo da aula preenchido: cadastro que rejeita e-mail duplicado, com quatro critérios. É o mesmo arquivo de onde a aula 4.3 parte.

No repositório: modulo-1-o-operador-e-o-trilho/aula-1.5-escolha-sua-feature-alvo/modelos/

`scripts/confere.sh`

O número na porta do próprio arquivo: aprova quando há 6 seções, nenhum espaço do modelo, de 3 a 10 critérios (acima de 5, avisa; acima de 10, reprova) e as 5 peneiras marcadas. Avisa palavra vaga. Aceita o formato `AC-NN` do Módulo 4.

No repositório: plugins/feature-alvo/skills/feature-alvo/scripts/

`modelos/pedido-feature-alvo.md`

O pedido da aula com as cinco peneiras dentro, para quem não instalou o plugin, e o que conferir na resposta do Claude.

No repositório: modulo-1-o-operador-e-o-trilho/aula-1.5-escolha-sua-feature-alvo/modelos/

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
`Plugin "feature-alvo" not found in marketplace "harness-lab"`	Marketplace adicionado numa aula anterior: a cópia local não tem o plugin novo, e o `add` não atualiza	`claude plugin marketplace update harness-lab` e o install de novo
`conflito FEATURE-ALVO.md` no `instalar.sh`	Você já tinha um `FEATURE-ALVO.md` na raiz	O seu ficou intacto. O modelo está em `FEATURE-ALVO.md.harness-lab-novo`: compare e apague o que sobrar
O `grep -c` nunca chega a 0	Tag ou código de verdade escrito sem crases (`<input>`) parece espaço do modelo	Ponha entre crases. O comando do passo 7 e o `confere.sh` ignoram o que está entre crases
O `confere.sh` reprova "critérios de aceite: 2" ou avisa "palavra vaga"	Faltou o caminho feliz, ou há adjetivo no lugar de resultado (rápido, melhor, amigável)	Acrescente o que não pode quebrar; troque o adjetivo por número ou texto exato
`git clean -fdn` lista `Would remove FEATURE-ALVO.md` (ou o arquivo sumiu depois de um `git clean -fd`)	O arquivo está solto: fora do git e fora do exclude	Commit ou `.git/info/exclude` (passo 8)

6 Você sai *com*

- [ ] `FEATURE-ALVO.md` na raiz e o comando do passo 7 dá 0. Com o plugin, o `confere.sh` do fim do `/feature-alvo` diz "resultado: ok".
- [ ] Uma feature que passou nas cinco peneiras, com a área medida: quantos arquivos falam dela e quantos são teste.
- [ ] De 3 a 5 critérios "Quando ..., então ...", cada um com um resultado que um teste consegue ler, incluindo o caminho feliz.
- [ ] O arquivo no commit ou no `.git/info/exclude`, nunca solto.
- [ ] Repositório escolhido: tem git, aceita branch de trabalho e o app ou os testes rodam aí.

PROMETIDO NA AULA

- ✓ **"No material de apoio tem uma tabela com mais exemplos de cada tipo, e a peneira em que cada um cai"**: seção 2, páginas 3 e 4, e `plugins/feature-alvo/skills/feature-alvo/references/exemplos.md`.
- ✓ **"Pega esses slides daqui para trás, joga no Claude e fala assim: eu quero encontrar uma feature-alvo a partir dessa peneira aqui. Me ajuda a encontrar"**: a skill `/feature-alvo` (seção 3, passos 3 e 4) e, sem plugin, o pedido do passo 5, também em `modelos/`.
- ✓ **"Ela é uma ideia num arquivo, no seu FEATURE-ALVO.md"**: o modelo que o `instalar.sh 1.5` põe na raiz (seção 3, passo 2) e o exemplo preenchido em `modelos/`, com um trecho no fim da seção 3.
- ✓ **"Você sai daqui com ... o repositório escolhido"**: a caixa sobre o repositório, na seção 2.
- ✓ **"A gente lê o log para cada um desses três casos"**: aula 6.2, Os 3 desfechos, no Módulo 6.

7 *Glossário*

Feature-alvo	A feature do seu repositório que o loop entrega no dia 7. Escolhida nesta aula, escrita no `FEATURE-ALVO.md`.
Critério de aceite	Uma linha que diz o que é certo, de um jeito que outra pessoa, ou um teste, confere sem te perguntar.
Isolada e acoplada	Isolada encosta em pouca coisa (um endpoint, um campo, uma validação). Acoplada encosta no sistema inteiro (ORM, autenticação, framework).
Terceiro instável	Serviço fora do seu controle que falha ou responde diferente: API externa, sandbox de pagamento, sistema de outro time, modelo de IA.
Modo de plano	Modo do Claude Code em que ele lê, explora e propõe, sem editar até você aprovar. Shift+Tab alterna.

8 Para *ir além*

- Best practices: "Let Claude run without you", as cinco perguntas antes de deixar uma tarefa rodando sozinha.
- Permission modes: "Analyze before you edit with plan mode", entrar, aprovar e sair.
- gitignore: quando usar `.gitignore`, `info/exclude` ou o ignore global.
- BC Harness: o `commands/plan.md`, base do `/plan FEATURE-ALVO.md` da aula 4.3.

Próxima: *Aula 2.1 · Hooks: o que o modelo não decide*

O Módulo 2 começa pela Trava, a primeira vitória: um hook que roda fora do modelo, sem ele decidir nada. Lembra do `migrate:fresh` da aula 1.3, a regra que o agente sabia e quebrou mesmo assim? Na aula 2.3 ele vai ser barrado.